

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ
КОМП'ЮТЕРНИХ СИСТЕМ**

Завдання 5: “Створення автоматизованих тестів”

Виконав:

ст. гр. КІ-404

Салагуб А.О.

Приняв:

Федак П.Р.

ЗАВДАННЯ

1. Впровадити або використовувати існуючу тестову бібліотеку;
2. Створити набір автоматизованих тестів;
3. Звіт про тестування повинен містити кількість усіх тестів, складених тестів, нескладених тестів, покриття;
4. Покриття повинно бути більше 80%
5. Необхідні кроки

Варіант 18.

Student number	Game	config format
18	rock paper scissors	INI

ТЕОРИТИЧНИЙ МАТЕРІАЛ

Автоматизовані тести — це програмні засоби, що автоматично виконують перевірку роботи програмного забезпечення (ПЗ) для забезпечення його якості. Автоматизація тестування має кілька переваг:

- *Швидкість виконання*: автоматичні тести працюють швидше за ручні.
- *Повторюваність*: тести можуть виконуватися з однаковими параметрами стільки разів, скільки потрібно.
- *Точність*: зменшується ймовірність людських помилок.
- *Покращення якості*: регулярне тестування дозволяє швидко виявити помилки та впровадити виправлення.
-

Види автоматизованих тестів

1. *Модульні тести (Unit Tests)*: перевіряють окремі компоненти або функції.
2. *Інтеграційні тести*: перевіряють взаємодію між компонентами.
3. *Системні тести*: оцінюють роботу ПЗ у цілому.
4. *Регресійні тести*: перевіряють, чи виправлення одного компонента не порушило роботу інших.

Google Test

Google Test (GTest) — це популярна бібліотека для модульного тестування в мові програмування C++, створена компанією Google. Вона забезпечує зручний інструментарій для створення та виконання тестів, що спрощує процес розробки та підвищує якість програмного забезпечення.

Основні можливості Google Test:

1. *Модульне тестування*: GTest дозволяє тестувати окремі функції, класи або модулі для забезпечення їх правильної роботи.
2. *Асерти (Assertions)*: спеціальні макроси для перевірки умов у тестах. Існує два типи асертів:
 - **ASSERT_***: зупиняє виконання тесту, якщо перевірка не пройдена.
 - **EXPECT_***: дозволяє тесту продовжуватися навіть при невдалому результаті перевірки.
3. *Тестові фікстури*: забезпечують спільне середовище для кількох тестів, що дозволяє уникнути дублювання коду (наприклад, ініціалізація або очищення ресурсів).
4. *Параметризовані тести*: дозволяють виконувати один і той самий тест із різними наборами вхідних даних.
5. *Інтеграція з системами CI/CD*: GTest легко інтегрується з платформами безперервної інтеграції (GitHub Actions, Jenkins, Travis CI тощо), що дозволяє автоматизувати тестування.
6. *Докладні звіти*: результати виконання тестів відображаються у зручному текстовому форматі, або у вигляді XML для аналізу іншими інструментами.

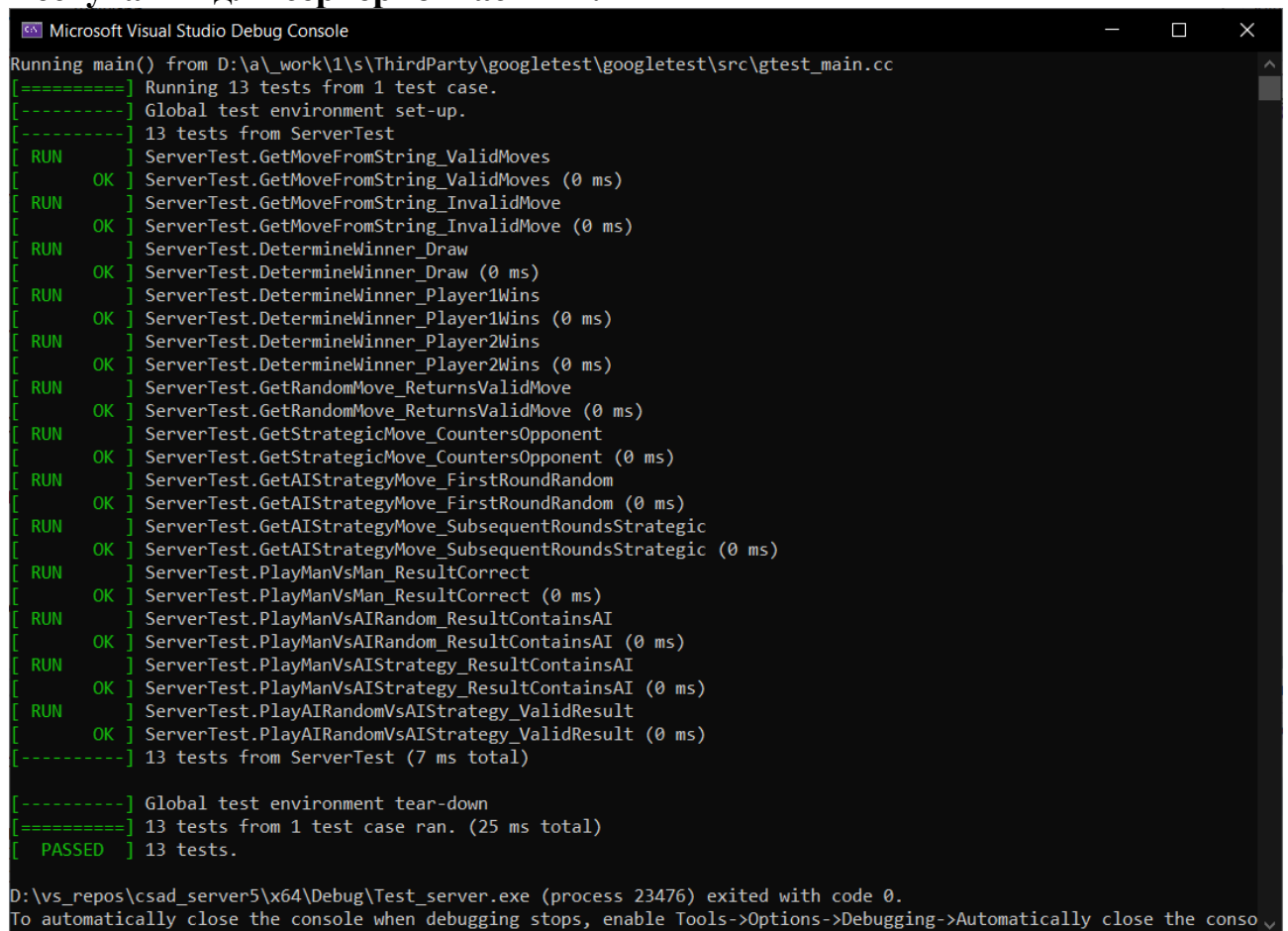
Покриття коду (Code Coverage)

Покриття коду — це метрика, яка показує, яка частина коду програми була виконана під час виконання тестів. Вимірювання покриття коду дозволяє зрозуміти, наскільки добре тести перевіряють програму, і допомагає знайти ділянки коду, які не тестуються.

ВИКОНАННЯ РОБОТИ

1. Для створення тестів використовувалася тестова бібліотека Google Test, яка була інтегрована у Visual Studio.
2. Були створені тести для серверної та клієнтської частини. А саме у файлах **test_game.cpp**, **test_serial.cpp** для клієнтської частини, та у файлі **test.cpp** для серверної частини
3. Звітність про кількість усіх тестів, складених тестів, нескладених тестів, покриття;

Тестування для серверної частини:



```
Microsoft Visual Studio Debug Console
Running main() from D:\a_work\1\s\ThirdParty\googletest\googletest\src\gtest_main.cc
[=====] Running 13 tests from 1 test case.
[-----] Global test environment set-up.
[-----] 13 tests from ServerTest
[ RUN ] ServerTest.GetMoveFromString_ValidMoves
[ OK ] ServerTest.GetMoveFromString_ValidMoves (0 ms)
[ RUN ] ServerTest.GetMoveFromString_InvalidMove
[ OK ] ServerTest.GetMoveFromString_InvalidMove (0 ms)
[ RUN ] ServerTest.DetermineWinner_Draw
[ OK ] ServerTest.DetermineWinner_Draw (0 ms)
[ RUN ] ServerTest.DetermineWinner_Player1Wins
[ OK ] ServerTest.DetermineWinner_Player1Wins (0 ms)
[ RUN ] ServerTest.DetermineWinner_Player2Wins
[ OK ] ServerTest.DetermineWinner_Player2Wins (0 ms)
[ RUN ] ServerTest.GetRandomMove_ReturnsValidMove
[ OK ] ServerTest.GetRandomMove_ReturnsValidMove (0 ms)
[ RUN ] ServerTest.GetStrategicMove_CountersOpponent
[ OK ] ServerTest.GetStrategicMove_CountersOpponent (0 ms)
[ RUN ] ServerTest.GetAIStrategyMove_FirstRoundRandom
[ OK ] ServerTest.GetAIStrategyMove_FirstRoundRandom (0 ms)
[ RUN ] ServerTest.GetAIStrategyMove_SubsequentRoundsStrategic
[ OK ] ServerTest.GetAIStrategyMove_SubsequentRoundsStrategic (0 ms)
[ RUN ] ServerTest.PlayManVsMan_ResultCorrect
[ OK ] ServerTest.PlayManVsMan_ResultCorrect (0 ms)
[ RUN ] ServerTest.PlayManVsAIRandom_ResultContainsAI
[ OK ] ServerTest.PlayManVsAIRandom_ResultContainsAI (0 ms)
[ RUN ] ServerTest.PlayManVsAIStrategy_ResultContainsAI
[ OK ] ServerTest.PlayManVsAIStrategy_ResultContainsAI (0 ms)
[ RUN ] ServerTest.PlayAIRandomVsAIStrategy_ValidResult
[ OK ] ServerTest.PlayAIRandomVsAIStrategy_ValidResult (0 ms)
[-----] 13 tests from ServerTest (7 ms total)

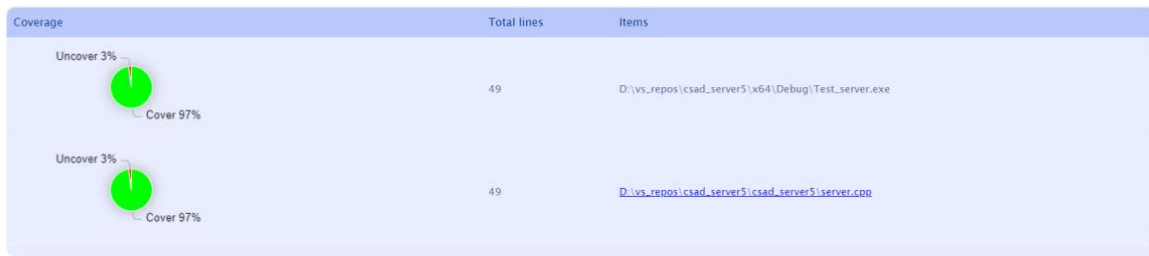
[-----] Global test environment tear-down
[=====] 13 tests from 1 test case ran. (25 ms total)
[ PASSED ] 13 tests.

D:\vs_repos\csad_server5\x64\Debug\Test_server.exe (process 23476) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
```

Рис. 1. Проходження тестів для серверної частини

Для серверної частини програми було написано 13 тестів, які були успішно пройдені. Покриття тестами коду сервера становить 97%. Результати з дослідження покриття коду знаходять у папці coverage, де містяться html файли з результатами.

Test_server.exe



Generated by [OpenCppCoverage \(Version: 0.9.9.0\)](#)

Тестування клієнтської частини:

```
Microsoft Visual Studio Debug Console
Running main() from D:\a\work\1\s\ThirdParty\googletest\googletest\src\gtest_main.cc
[=====] Running 18 tests from 2 test cases.
[-----] Global test environment set-up.
[-----] 5 tests from GameTests
[ RUN    ] GameTests.SaveGameState
[ OK     ] GameTests.SaveGameState (7 ms)
[ RUN    ] GameTests.LoadGameState
[ OK     ] GameTests.LoadGameState (2 ms)
[ RUN    ] GameTests.PlayGame_ManVsMan
[ OK     ] GameTests.PlayGame_ManVsMan (1 ms)
[ RUN    ] GameTests.PlayGame_ManVsAI
[ OK     ] GameTests.PlayGame_ManVsAI (2 ms)
[ RUN    ] GameTests.PlayGame_Exit
[ OK     ] GameTests.PlayGame_Exit (7 ms)
[-----] 5 tests from GameTests (24 ms total)

[-----] 13 tests from SerialPortTest
[ RUN    ] SerialPortTest.ConstructorOpensPortAndConfigures
[ OK     ] SerialPortTest.ConstructorOpensPortAndConfigures (21 ms)
[ RUN    ] SerialPortTest.SendMessageTest
[ OK     ] SerialPortTest.SendMessageTest (30 ms)
[ RUN    ] SerialPortTest.ReceiveMessageTest
[ OK     ] SerialPortTest.ReceiveMessageTest (10 ms)
[ RUN    ] SerialPortTest.DestructorClosesPort
Unable to open port: COM3
[ OK     ] SerialPortTest.DestructorClosesPort (10 ms)
[ RUN    ] SerialPortTest.SetPortStateTest
[ OK     ] SerialPortTest.SetPortStateTest (15 ms)
[ RUN    ] SerialPortTest.SetPortTimeoutsTest
[ OK     ] SerialPortTest.SetPortTimeoutsTest (15 ms)
[ RUN    ] SerialPortTest.ConstructorOpensValidPort
[ OK     ] SerialPortTest.ConstructorOpensValidPort (11 ms)
[ RUN    ] SerialPortTest.ConstructorFailsToOpenInvalidPort
Unable to open port: INVALID_PORT
[ OK     ] SerialPortTest.ConstructorFailsToOpenInvalidPort (1 ms)
[ RUN    ] SerialPortTest.SendMessageValidMessage
[ OK     ] SerialPortTest.SendMessageValidMessage (30 ms)
[ RUN    ] SerialPortTest.SendMessageEmptyMessage
[ OK     ] SerialPortTest.SendMessageEmptyMessage (11 ms)
[ RUN    ] SerialPortTest.ReceiveMessageNoResponse
[ OK     ] SerialPortTest.ReceiveMessageNoResponse (10305 ms)
[ RUN    ] SerialPortTest.SetPortStateSuccess
[ OK     ] SerialPortTest.SetPortStateSuccess (16 ms)
[ RUN    ] SerialPortTest.SetPortTimeoutsSuccess
[ OK     ] SerialPortTest.SetPortTimeoutsSuccess (17 ms)
[-----] 13 tests from SerialPortTest (10515 ms total)

[-----] Global test environment tear-down
[=====] 18 tests from 2 test cases ran. (10541 ms total)
```

Рис. 2. Проходження тестів клієнтом

Для клієнтської частини було створенно 18 тестів, які були успішно пройдені. Покриття коду клієнта тестами становить 78%. На скріні зображено,

які функції були протестованими. Результати з дослідження покриття коду знаходять у папці coverage, де містяться html файли з результатами.

Test_sample.exe

Coverage	Total lines	Items
 Uncover 22% Cover 78%	211	D:\vs_repos\csad_lab3_new_menu\x64\Debug\Test_sample.exe
 Uncover 21% Cover 79%	153	D:\vs_repos\csad_lab3_new_menu\csad_lab3_new_menu\game.cpp
 Uncover 25% Cover 75%	58	D:\vs_repos\csad_lab3_new_menu\csad_lab3_new_menu\serialport.cpp

Generated by [OpenCppCoverage \(Version: 0.9.9.0\)](#)

Приклад частини тестів(клієнт):

```
TEST(GameTests, SaveGameState) {
    std::string filename = "test_game.ini";
    std::string mode = "Man vs Man";
    std::string player1Move = "rock";
    std::string player2Move = "scissors";
    int player1Score = 1;
    int player2Score = 0;
    saveGameState(filename, mode, player1Move, player2Move, player1Score,
player2Score);
    std::ifstream inFile(filename);
    ASSERT_TRUE(inFile.is_open());

    std::string line;
    std::getline(inFile, line);
    ASSERT_EQ(line, mode);

    std::getline(inFile, line);
    ASSERT_EQ(line, player1Move);

    std::getline(inFile, line);
    ASSERT_EQ(line, player2Move);

    std::getline(inFile, line);
    ASSERT_EQ(std::stoi(line), player1Score);

    std::getline(inFile, line);
    ASSERT_EQ(std::stoi(line), player2Score);

    inFile.close();
    std::remove(filename.c_str());
}

TEST(GameTests, LoadGameState) {
    std::string filename = "test_game.ini";
    std::ofstream outFile(filename);
    outFile << "Man vs Man\n";
    outFile << "rock\n";
    outFile << "scissors\n";
    outFile << 1 << "\n";
    outFile << 0 << "\n";
    outFile.close();
    std::string mode, player1Move, player2Move;
```

```
    int player1Score, player2Score;
    bool result = loadGameState(filename, mode, player1Move, player2Move,
player1Score, player2Score);
    ASSERT_TRUE(result);
    ASSERT_EQ(mode, "Man vs Man");
    ASSERT_EQ(player1Move, "rock");
    ASSERT_EQ(player2Move, "scissors");
    ASSERT_EQ(player1Score, 1);
    ASSERT_EQ(player2Score, 0);
    std::remove(filename.c_str());
}
```

ВИСНОВОК

Під час виконання лабораторної роботи успішно реалізовано всі заплановані завдання. Було протестовано клієнтську і серверну частину гри камінь-ножниця-папір. Загалом було написано 31 тест, кожен з яких успішно складався. Покриття кодом клієнтської частини 78%, а серверної 97%. Якщо говорити про всю програму то покриття тестами в середньому приблизно становить 85%.

Результати з дослідження покриття коду знаходять у папці coverage, де містяться html файли з результатами. Також опанував необхідні навички для тестування програм за допомогою GTests.

СПИСОК ЛІТЕРАТУРИ

1. Google Test Documentation: <https://google.github.io/googletest/>
2. Beck, Kent. *Test-Driven Development: By Example*. — Addison-Wesley, 2022.
3. Fewster, Mark, Graham, Dorothy. *Experiences of Test Automation: Case Studies of Software Test Automation*. — Addison-Wesley, 2018.